

1 Grundlagen

Orientierung ca. 8-12 Seiten

1.1 Grundlagen der Hintergrundverarbeitung

Hintergrundverarbeitung dient dazu, zeitintensive oder ressourcenintensive Aufgaben vom synchronen Request-Response-Zyklus zu entkoppeln. Gleichzeitig entstehen daraus jedoch auch einige Herausforderungen: In verteilten Umgebungen sind Teilausfälle, Timeouts und kurzzeitige Unterbrechungen üblich, sodass nicht ohne entsprechende Mechanismen davon ausgegangen werden kann, dass eine angestoßene Verarbeitung vollständig und genau einmal ausgeführt wird. Insbesondere kann bei Kommunikationsabbrüchen oder Wiederholungsversuchen unklar bleiben, ob eine Operation bereits verarbeitet wurde oder ob sie erneut angestoßen werden muss (Kleppmann 2017, Kap. 8). Vor diesem Hintergrund werden in den folgenden Abschnitten grundlegende Konzepte vorgestellt, die eine Entkopplung der Verarbeitung ermöglichen und die Basis für robuste Hintergrundverarbeitung bilden.

1.1.1 Synchrone vs. Asynchrone Verarbeitung

In Backend-Systemen lässt sich Kommunikation und Verarbeitung bezüglich der zeitlichen Kopplung zwischen synchronem und asynchronem Verhalten unterscheiden: Bei synchroner Verarbeitung nach dem Request-Response-Stil wartet der Aufrufer auf eine Antwort auf die Anfrage und blockiert gegebenenfalls bevor er fortfährt. Bei asynchroner Verarbeitung wird die Bearbeitung des Auftrags vom Zeitpunkt des Erhalts entkoppelt. Der Aufrufer initiiert die Arbeit, blockiert jedoch nicht und kann auch während des Wartens auf eine Antwort weiter arbeiten (Richardson 2018, Kap. 3.1.1).

Um Nachrichtenverluste zu reduzieren und Lastspitzen auszugleichen, verwenden asynchrone Systeme in der Praxis häufig einen Puffer. Durch das sogenannte „queueing“ (Kleppmann 2017, Kap. 11, Abschnitt „Message brokers“), bei dem erhaltene Anfragen zur späteren Ausführung in den Puffer geschrieben werden, wird unter anderem die Asynchronität erreicht. Ein Produzent sendet eine Nachricht an einen Zwischenspeicher, beispielsweise einen Message Broker, und wartet in der Regel nur auf das Acknowledgment, dass die Nachricht zwischengespeichert wurde, nicht jedoch auf die Verarbeitung oder das Ergebnis (Kleppmann 2017, Kap. 11, Abschnitt „Messaging Systems“). Die so erreichte Entkopplung hat gleichzeitig den Vorteil, dass die Verarbeitung der Anfrage unabhängig von der Verbindung zwischen Produzent und Konsument wird. Im Gegensatz dazu kann bei synchronen Systemen bei Verbindungsabbrüchen oder Timeouts unklar bleiben, ob eine Anfrage den Server erreicht hat oder bereits verarbeitet wurde. Genau dieses Problem adressieren Message Broker: Falls ein Konsument temporär nicht verfügbar ist oder während der Verarbeitung ausfällt, können Message Broker trotzdem für die At-Least-Once-Semantik (vgl. Kapitel 1.1.2) sorgen (Kleppmann 2017, Kap. 11, Abschnitt „Ack-

nowledgments and redelivery“). Die synchrone Request-Response-Kommunikation bietet demgegenüber eine unmittelbare Rückmeldung über Erfolg oder Fehler einer Operation, setzt jedoch voraus, dass Client und Server während der gesamten Interaktion verfügbar sind (Richardson 2018, Kap. 3.4).

Die asynchrone Verarbeitung bildet damit die Grundlage für die Hintergrundverarbeitung, indem die Entkopplung genutzt wird, um Aufgaben unabhängig vom ursprünglichen Aufruf auszuführen und gleichzeitig die Robustheit und Skalierbarkeit des Systems zu erhöhen (Newman 2026, Kap. 8, Abschnitt „Why Use A Message Broker?“).

1.1.2 Ausführungssemantiken

In verteilten Systemen führen potenziellen Prozessabstürze und Teilausfälle dazu, dass Aufträge verloren gehen oder der Verarbeitungszustand einer Aufgabe unklar sein kann. Um zu beschreiben, was Systeme aufgrund ihres Entwurfs und ihrer Mechanismen zur Fehlertoleranz bezüglich Auslieferung beziehungsweise Ausführung garantieren, werden grundsätzlich drei fundamentale Ausführungssemantiken unterschieden, die Newman (2026, Kap. 8 Abschnitt „Types of Delivery Guarantees“) vor dem Hintergrund der Auslieferung wie folgt definiert. Im Kontext von Job-Processing sind diese Semantiken zur Nachrichtenzustellung analog auf die Ausführung der Jobs anwendbar.

At-Least-Once

Ein Konsument erhält die Nachricht mindestens einmal. Bei der Auslieferung bestätigt der Konsument über ein Acknowledge das Ankommen der Nachricht. Sofern das Acknowledge beim Absender eingeht, kann dieser die Nachricht löschen, bleibt diese Bestätigung innerhalb eines Zeitfensters jedoch aus, z.B. weil die Nachricht oder das Acknowledge verloren geht, wird die Nachricht erneut zugestellt. Dieser Fall wird Redelivery genannt. Dieses Verfahren verhindert Datenverlust, birgt jedoch das Risiko von Duplikaten, für den Fall, dass die Nachricht ankommt, aber das Acknowledge verloren geht und es infolgedessen zur Redelivery kommt. Einschränkend sei hier gesagt, dass die Zustellung nicht garantiert werden kann, wenn der Konsument bei keinem Zustellversuch verfügbar ist.

At-Most-Once

Ein Konsument erhält die Nachricht maximal einmal. Der Produzent schickt die Nachricht ab und erwartet keine Empfangsbestätigung. Eine erneute Zustellung findet unter keinen Umständen statt. Dadurch werden Duplikate strikt ausgeschlossen, jedoch muss ein potenzieller Datenverlust zugunsten besserer Latenzen hingenommen werden.

Exactly-Once

Diese Semantik garantiert, dass eine Nachricht genau einmal beim Konsumenten ankommt. In der Praxis ist das nur schwer zu realisieren, da beispielsweise nicht klar sein kann, ob ein Acknowledgement nur noch nicht angekommen, oder schon verloren gegangen ist. Viele Umsetzungen beruhen daher darauf, dass entsprechende Mechanismen

sowohl beim Produzent, als auch Konsument implementiert werden. Eine Variante zur Umsetzung ist die Approximation über eine Kombination von At-Least-Once beim Produzent und Idempotenz beim Konsument. Die Garantie bezieht sich somit nicht auf die physische Zustellung einer Nachricht, sondern auf die Wirkung ihrer Verarbeitung. Eine Nachricht kann mehrfach zugestellt werden, solange ihre Verarbeitung nur einmal wirksam wird.

1.1.3 Background Job-Processing

Aufbauend auf den Konzepten der asynchronen Verarbeitung (vgl. Kapitel 1.1.1) bezeichnet das Background Job-Processing die Ausführung von Aufgaben außerhalb des Request-Response-Zyklus. Hierbei werden fachliche Operationen als eigenständige Arbeitseinheiten (Jobs) modelliert und zeitlich entkoppelt verarbeitet.

In der Softwarearchitektur existieren verschiedene etablierte Entwurfsmuster zur Umsetzung einer solchen Infrastruktur, wie beispielsweise das Queue-Based Load Leveling Pattern (Microsoft Corporation (Hrsg.) 2026b). Dieses Muster besteht aus drei funktionalen Kernkomponenten:

- **Der Produzent:** Erzeugt einen Job und gibt diesen weiter an den Job-Store. Das kann zum einen direkt durch das User Interface (UI) geschehen, oder die UI ruft einen Endpoint auf, der den Job über eine synchrone Schnittstelle, wie z.B. Representational State Transfer (REST), entgegen nimmt. In letzterem Fall fungiert der Service hinter dem Endpoint als Produzent. Dessen Aufgabe besteht darin, den Auftrag zu validieren und an die Persistenzschicht zu übergeben, woraufhin der synchrone Request-Response-Zyklus für den Client sofort beendet wird. Jobs können aber nicht nur durch solche Event-driven Trigger ausgelöst werden, sondern auch durch Schedule-driven Trigger. Dabei erfolgt die Ausführung eines Jobs periodisch oder zu vordefinierten Zeitpunkten, z.B. über durch einen Cron-Trigger (Microsoft Corporation (Hrsg.) 2026a).
- **Der Job-Store:** Speichert die Jobs für den Zeitraum zwischen Erstellung und endgültiger Beendigung zwischen. Für die Umsetzung existieren verschiedene Varianten, beispielsweise als Message Queue oder relationale Datenbank. Diese Realisierungsmöglichkeiten bieten jeweils spezifische Vor- und Nachteile und die Nutzung. Die Auswahl des passenden Ansatzes sollte daher durch die Qualitätsanforderungen an das System, wie z.B. Durchsatz oder Akzeptanz einzelner verlorener Jobs, begründet werden (Kleppmann 2017, Kap. 11).
- **Der Worker:** Autonome Komponente, die als Konsument fungiert. Sobald der Worker über freie Systemressourcen verfügt, kann er einen wartenden Job aus dem Job-Store beanspruchen und führt diesen aus (Microsoft Corporation (Hrsg.) 2026b).

Durch eine solche Architektur können Background Jobs als „fire and forget“-Operationen (Microsoft Corporation (Hrsg.) 2026a) implementiert werden. Sie laufen asynchron in einem vom Client isolierten Prozess, sodass deren Verarbeitungszeit keinerlei unmittelbare

Auswirkung auf die Reaktionszeit oder Verfügbarkeit der primären Benutzeroberfläche hat (Microsoft Corporation (Hrsg.) 2026a). Außerdem ermöglicht dieses Konzept das Abfangen von Lastspitzen und flexiblere Skalierung (Microsoft Corporation (Hrsg.) 2026b).

Die Zuweisung von Jobs an Worker kann grundsätzlich über unterschiedliche Modelle erfolgen. Beim Push-Modell weist eine zentrale Komponente Aufgaben aktiv freien Worker-Instanzen zu. Beim Pull-Modell fragen Worker eigenständig neue Aufgaben aus dem Job-Store ab. Beide Ansätze unterscheiden sich hinsichtlich Komplexität, Skalierbarkeit und Koordinationsaufwand (Kleppmann 2017, Kap. 11 Abschnitt „Transmitting Event Streams“).

Da die Verarbeitung zeitlich entkoppelt erfolgt, müssen Informationen über offene und bereits bearbeitete Jobs über den Lebenszyklus einzelner Prozesse hinweg erhalten bleiben. Hierfür ist eine persistente Speicherung des Verarbeitungszustands erforderlich, da das System nach einem Ausfall in der Lage sein muss, den zuletzt bekannten Zustand zu rekonstruieren und die Verarbeitung fortzusetzen (Kleppmann 2017, Kap. 11, Abschnitt „Rebuilding state after a failure“). Grundlagen zur Zustandshaltung werden in Kapitel 1.4 betrachtet.

Werden mehrere Worker parallel betrieben, entsteht die Herausforderung der konkurrierenden Verarbeitung. Ohne geeignete Koordinationsmechanismen könnten mehrere Instanzen denselben Job gleichzeitig beanspruchen oder widersprüchliche Statusänderungen durchführen. Die hierfür relevanten Konzepte werden in Kapitel 1.2 behandelt.

Darüber hinaus müssen Background-Processing-Systeme mit Teilausfällen umgehen können. Prozesse können während der Ausführung terminieren, externe Systeme zeitweise nicht erreichbar sein oder Kommunikationsfehler auftreten. Das System muss daher entscheiden können, ob eine Verarbeitung erneut durchgeführt, fortgesetzt oder endgültig beendet werden soll (Microsoft Corporation (Hrsg.) 2026a). Konzepte zur Fehlerbehandlung und Wiederherstellung werden in Kapitel 1.3 vorgestellt.

Ebenfalls kritisch ist die Frage, wie sichergestellt werden kann, dass Wiederholungen oder Mehrfachausführungen nicht zu inkonsistenten Ergebnissen führen. Da der tatsächliche Verarbeitungszustand nach einem Fehler nicht immer eindeutig bestimmbar ist, müssen Systeme geeignete Konsistenzmechanismen bereitstellen. Hierzu zählen insbesondere Transaktionskonzepte und Verfahren zur kontrollierten Wiederaufnahme von Verarbeitungen, die in Kapitel 1.3 behandelt werden.

1.2 Konsistenz und konkurrierende Verarbeitung

Bei der Verarbeitung von Daten durch mehrere parallel arbeitende Prozesse entsteht die Herausforderung, dass konkurrierende Zugriffe auf gemeinsame Ressourcen zu inkonsistenten Zuständen führen können. Ohne geeignete Mechanismen können Änderungen verloren gehen, widersprüchliche Zustände entstehen oder mehrere Prozesse dieselben Daten gleichzeitig verändern (Saake, Heuer und Sattler 2018, Kap. 13.1.2).

Im Kontext von Background Job-Processing tritt dieses Problem insbesondere dann auf, wenn mehrere Worker-Instanzen parallel auf denselben Jobbestand zugreifen. Versuchen sie ohne geeignete Synchronisationsmechanismen denselben Job zu beanspruchen

oder konkurrierende Statusänderungen durchzuführen, können Race Conditions entstehen (Kleppmann 2017, Kap. 7 Abschnitt „Isolation“). Insbesondere bei Pull-basierten Architekturen, bei denen die Jobverteilung nicht durch eine Orchestrierungskomponente realisiert wird, muss eine solche ungewollte Doppelbeanspruchung explizit verhindert werden (Kleppmann 2017, Kap. 7 Abschnitt „Isolation“). Die Gewährleistung konsistenter Zustände stellt daher eine zentrale Anforderung an Systeme mit paralleler Verarbeitung dar. Die hierfür etablierten Konzepte und Mechanismen werden in den folgenden Abschnitten vorgestellt.

1.2.1 ACID

Zur Vereinfachung der Gewährleistung konsistenter Zustände bei parallelen Zugriffen existiert das Konzept der Transaktionen. Diese fassen mehrere Operationen zu einer logischen Einheit zusammen, deren Ausführung nach außen als zusammenhängender Vorgang erscheint (Kleppmann 2017, Kap. 7). Im Kontext persistenter Datenhaltung werden von Transaktionen typischerweise die vier zentralen Eigenschaften Atomicity, Consistency, Isolation, Durability (ACID) gefordert (Saake, Heuer und Sattler 2018, Kap. 13.1.1):

Atomicity Die Atomarität stellt sicher, dass eine Transaktion entweder vollständig oder gar nicht ausgeführt wird. Tritt während der Ausführung ein Fehler auf, werden sämtliche bis dahin vorgenommenen Änderungen verworfen, sodass keine partiellen Zwischenzustände bestehen bleiben.

Consistency Die Konsistenz beschreibt die Eigenschaft, dass eine erfolgreich abgeschlossene Transaktion die Daten von einem gültigen Zustand in einen anderen gültigen Zustand überführt. Dabei müssen definierte Integritätsbedingungen erhalten bleiben. Kleppmann (2017, Kap. 7 Abschnitt „Consistency“) weist darauf hin, dass die Einhaltung solcher fachlichen Invarianten in der Verantwortung der Anwendung liegt und nicht durch das Datenbanksystem garantiert werden kann.

Isolation Die Isolation gewährleistet, dass parallel ausgeführte Transaktionen sich nicht gegenseitig beeinflussen. Aus Sicht einer Transaktion soll die Ausführung so erscheinen, als würde sie allein auf dem Datenbestand arbeiten. Dadurch können Race Conditions und andere Nebenläufigkeitsprobleme vermieden werden.

Durability Die Dauerhaftigkeit garantiert, dass die Ergebnisse einer erfolgreich abgeschlossenen Transaktion auch nach Systemabstürzen oder anderen Fehlern dauerhaft erhalten bleiben.

Im Kontext von Background Job-Processing sind insbesondere die Eigenschaften der Atomarität und Isolation von Bedeutung. Während die Atomarität verhindert, dass Statusänderungen eines Jobs nur teilweise durchgeführt werden, bildet die Isolation die Grundlage für die koordinierte Verarbeitung gemeinsamer Datenbestände durch mehrere parallel arbeitende Worker-Instanzen. Die relevanten Konzepte zum Erreichen dieser Eigenschaften werden in den folgenden Abschnitten näher betrachtet.

1.2.2 Transaktionsgrenzen

Während ACID die theoretischen Garantien einer Transaktion definiert, bestimmt die Festlegung von Transaktionsgrenzen, welche konkreten Datenbankoperationen des softwareseitigen Kontrollflusses zu einer unteilbaren, logischen Einheit zusammengefasst werden. Die Festlegung dieser Grenzen steuert, wann eine Datenbanksitzung eine Transaktion initiiert und zu welchem Zeitpunkt die Änderungen final festgeschrieben (Commit) oder im Fehlerfall vollständig verworfen (Rollback) werden (Kudraß 2015, S.250).

Die Wahl geeigneter Transaktionsgrenzen wird zusätzlich dadurch erschwert, dass fachliche und technische Transaktionen nicht zwangsläufig identisch sind. Eine fachliche Transaktion beschreibt eine zusammenhängende Geschäftsoperation, deren Ergebnis aus Sicht der Anwendung als Einheit betrachtet wird. Eine technische Transaktion hingegen bezeichnet die konkrete Transaktion auf der Datenbankebene, welche die ACID-Eigenschaften für die von ihr gekapselten Datenbankoperationen bereitstellt. Während eine fachliche Operation mehrere technische Transaktionen umfassen kann, können innerhalb einer technischen Transaktion auch nur Teilaspekte einer fachlichen Operation verarbeitet werden. Werden die Grenzen falsch gesetzt, können verschiedene Parallelitätsanomalien auftreten (Saake, Heuer und Sattler 2018, Kap. 13.1.2). Ein verbreiteter Ansatz zur Vermeidung dieser Anomalien ist die Verwendung von Sperren, auf die in Kapitel 1.2.3 näher eingegangen wird. Wichtig ist an dieser Stelle, dass die gewählte Transaktionsgrenze direkten Einfluss auf die Dauer von Sperren und die Interaktion mit parallel ausgeführten Transaktionen hat.

Eine Möglichkeit besteht darin, jede Datenbankoperation in einer eigenen Transaktion auszuführen. Dadurch werden Sperren nur für sehr kurze Zeit gehalten, was die Parallelität und den Durchsatz des Systems erhöht. Allerdings können zusammengehörige Änderungen nicht mehr atomar ausgeführt werden. Tritt zwischen mehreren aufeinanderfolgenden Operationen ein Fehler auf, können bereits festgeschriebene Änderungen nicht mehr gemeinsam zurückgesetzt werden. Die betroffenen Daten können dadurch in einen fachlich falschen Zustand überführt werden.

Als naiver Gegenentwurf können die Transaktionsgrenzen entsprechend der fachlichen Transaktion gesetzt werden. Diese Strategie kann jedoch zu langlaufenden Transaktionen führen, welche die Leistungsfähigkeit des Datenspeichers beeinträchtigt. Zum einen halten Transaktionen im Beispiel des 2-Phasen-Sperrprotokolls sämtliche während ihrer Laufzeit akquirierten Sperren bis zum endgültigen Commit oder Rollback aufrecht (Kleppmann 2017, Kap. 7 Abschnitt „Two-Phase Locking (2PL)“). Dadurch steigt die Wahrscheinlichkeit, dass konkurrierende Transaktionen aufeinander warten müssen. Zum anderen können langlaufende Transaktionen die Bereinigung alter Datenversionen verzögern, da diese weiterhin für aktive Transaktionen sichtbar bleiben müssen (The PostgreSQL Global Development Group (Hrsg.) 2026, Kap. 24.1.2). Dies erhöht den Ressourcenbedarf des Datenbanksystems und kann dessen Leistungsfähigkeit beeinträchtigen.

Im Software-Entwurf erfordert das Setzen der Transaktionsgrenzen daher eine kontinuierliche Abwägung zwischen der Systemperformance und dem Schutz vor Datenanomalien. Weit gefasste Transaktionsgrenzen reduziert das Risiko der Entstehung von Inkonsistenzen, blockieren jedoch Systemressourcen und Datensätze. Eng gefasste Grenzen

erhöhen den Durchsatz, können jedoch das Risiko konkurrierender Zugriffe und daraus resultierender Race Conditions erhöhen.

1.2.3 Synchronisationsverfahren

Zur Umsetzung der Isolationseigenschaft von Transaktionen müssen konkurrierende Zugriffe auf gemeinsame Daten koordiniert werden. Zu diesem Zweck existieren verschiedene Synchronisationsverfahren, die mittels unterschiedlicher Strategien das Entstehen von Parallelitätsanomalien verhindern (Kudraß 2015, S. 253–259).

Je nach Anwendungskontext kann es sein, dass nicht alle dieser Anomalien auftreten können, beispielsweise wenn nur Leseoperationen ausgeführt werden, oder dass deren Auftreten in Kauf genommen werden kann, um eine bessere Performance zu erhalten. Datenbanksysteme stellen zu diesem Zweck unterschiedliche Isolationsstufen bereit. Diese legen fest, in welchem Umfang parallele Transaktionen die Änderungen anderer Transaktionen beobachten können und bestimmen somit, welche Anomalien auftreten können. Dadurch müssen weniger Überprüfungen stattfinden und weniger Transaktionen müssen zurückgesetzt werden, was zu einer besseren Performance führt. Höhere Isolationsstufen bieten stärkere Konsistenzgarantien, können jedoch die Parallelität des Systems verringern (Saake, Heuer und Sattler 2018, Kap. 13.1.3).

Eine verbreitete Strategie zur Durchsetzung der Isolation ist die pessimistische Synchronisation. Sie basiert auf der Annahme, dass konkurrierende Zugriffe auftreten können und verhindert diese bereits im Vorfeld durch Sperrmechanismen. Bevor eine Transaktion einen Datensatz verändert, wird dieser für andere konkurrierende Transaktionen gesperrt. Der Vorteil pessimistischer Verfahren besteht darin, dass Konflikte bereits vor ihrer Entstehung verhindert werden. Dadurch eignen sie sich insbesondere für Szenarien mit häufigen konkurrierenden Zugriffen. Nachteilig wirken sich jedoch zusätzliche Wartezeiten aus, da Transaktionen auf die Freigabe benötigter Sperren warten müssen. Bei hoher Auslastung kann dies die Parallelität und den Gesamtdurchsatz eines Systems reduzieren. Außerdem entsteht die Gefahr von Deadlocks (Kudraß 2015, S. 253–257).

Alternative Ansätze sind nicht sperrende Verfahren, beispielsweise die optimistische Synchronisation oder das Zeitstempelverfahren. Anstatt Daten vor der Bearbeitung zu sperren, werden bei der optimistischen Synchronisation Transaktionen zunächst ausgeführt und erst beim Schreiben geprüft, ob ein Konflikt entstanden ist. Ist es zwischen zwei oder mehreren Transaktionen zum Konflikt gekommen, müssen einzelne Transaktionen zurückgesetzt und wiederholt werden. Bei dem Zeitstempelverfahren werden Zeitstempel für jede Transaktionen, sowie der Zeitpunkt des letzten Lese- und Schreibzugriffs für jedes Datenbankobjekt verwaltet. Anhand dieser Zeitstempel kann geprüft werden, ob zwei Transaktionen in Konflikt zueinander stehen und ob eine der Transaktionen zurückgesetzt werden muss. Nicht sperrende Verfahren vermeiden die durch Sperren verursachten Wartezeiten und ermöglichen dadurch eine hohe Parallelität. Treten jedoch Konflikte auf, müssen betroffene Operationen wiederholt werden. Sie eignen sich daher insbesondere für Systeme, in denen konkurrierende Änderungen selten vorkommen (Kudraß 2015, S. 257–259).

Welche Synchronisationsstrategie geeignet ist, hängt von den Anforderungen des je-

weiligen Systems und der Häufigkeit erwarteter Konflikte ab. Während pessimistische Verfahren Konflikte präventiv verhindern, akzeptieren optimistische Verfahren mögliche Konflikte und behandeln diese nachträglich.

1.3 Fehlertoleranz und Wiederherstellung

Verteilte Softwaresysteme sind zwangsläufig mit einer Vielzahl potenzieller Fehlerquellen konfrontiert. Prozesse können unerwartet terminieren, Netzwerkverbindungen unterbrochen werden oder externe Dienste zeitweise nicht erreichbar sein (Newman 2026, Kap. 1 Abschnitt „The Three Golden Rules Of Distributed Systems“). Beispielsweise kann es bei einzelnen Teilausfällen dazu kommen, dass unklar bleibt, ob ein Verarbeitungsschritt erfolgreich abgeschlossen wurde oder erneut ausgeführt werden muss. Da sich solche Fehler in komplexen Systemlandschaften nicht vollständig vermeiden lassen, muss ihre Existenz als grundlegende Systemeigenschaft betrachtet werden (Newman 2026, Kap. 1 Abschnitt „Failure Is Inevitable“).

Die Zuverlässigkeit eines Systems ergibt sich daher nicht primär daraus, Fehler vollständig zu verhindern, sondern daraus, wie gut mit ihrem Auftreten umgegangen wird. Fehlertoleranz bezeichnet in diesem Zusammenhang die Fähigkeit eines Systems, seine wesentlichen Funktionen trotz auftretender Störungen weiterhin bereitzustellen. Darüber hinaus muss ein System in der Lage sein, nach einem Fehler einen konsistenten Zustand wiederherzustellen und die Verarbeitung kontrolliert fortzusetzen, beziehungsweise neuzustarten (Newman 2026, Kap. 1 „What Is Resilience?“). Die folgenden Abschnitte stellen die hierfür relevanten Konzepte und Mechanismen vor.

1.3.1 Fehlerklassifikation

Da Fehler in verteilten Systemen unvermeidbar sind, stellt sich nicht die Frage, ob sie auftreten, sondern wie mit ihnen umgegangen werden soll. Die Wirksamkeit von Fehlertoleranzmechanismen hängt dabei wesentlich davon ab, die Ursache und Eigenschaften eines Fehlers korrekt einzuordnen. Nicht jeder Fehler kann durch dieselben Maßnahmen behandelt werden. Während einige Fehler durch eine erneute Ausführung einer Operation behoben werden können, sind andere dauerhaft und erfordern eine Anpassung der Eingabedaten oder eine manuelle Intervention. Die Klassifikation von Fehlern bildet daher die Grundlage für Entscheidungen über Retry-Strategien, Fehlerbehandlung und Wiederherstellungsmechanismen (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Darüber hinaus können nicht alle denkbaren Fehler mit vertretbarem Aufwand behandelt werden. Während kurzzeitige Infrastrukturprobleme, Prozessabstürze oder Kommunikationsfehler häufig durch geeignete Architekturmechanismen abgefangen werden können, liegen andere Ereignisse, etwa großflächige Naturkatastrophen oder der vollständige Ausfall eines Rechenzentrums, außerhalb des typischen Fehlerannahmemodells vieler Anwendungen. Der Entwurf fehlertoleranter Systeme erfordert daher eine bewusste Entscheidung darüber, welche Fehlerarten berücksichtigt werden und welche Risiken akzeptiert werden müssen (Kleppmann 2017, Kap. 1 Abschnitt „Reliability“).

Eine grundlegende Einteilung erfolgt in transiente und permanente Fehler. Transiente Fehler treten zeitlich begrenzt auf und verschwinden nach kurzer Zeit wieder. Ursachen liegen häufig in der technischen Infrastruktur eines Systems, beispielsweise in vorübergehenden Netzwerkstörungen, kurzzeitigen Überlastungen von Diensten, nicht erreichbaren Datenbanken oder dem Neustart einzelner Systemkomponenten. In solchen Fällen kann eine erneute Ausführung einer Operation erfolgreich sein, obwohl der erste Versuch fehlgeschlagen ist. Aus diesem Grund eignen sich transiente Fehler besonders für den Einsatz von Retry-Mechanismen auf die näher in Kapitel 1.3.2 eingegangen wird (Newman 2026, Kap. 3 Abschnitt „Should You Always Retry?“).

Permanente Fehler hingegen bestehen unabhängig von der Anzahl der Wiederholungsversuche fort. Ihre Ursache liegt häufig in der Anwendung selbst oder in den zu verarbeitenden Daten. Beispiele hierfür sind ungültige Eingabedaten oder fehlende Berechtigungen. Newman (2026, Kap. 3 Abschnitt „Should You Always Retry?“) verdeutlicht dies anhand von HTTP-Fehlercodes wie 404 Not Found oder 403 Forbidden. Da sich die zugrunde liegende Ursache durch eine Wiederholung der Anfrage nicht verändert, führen Retries in diesen Fällen lediglich zu zusätzlicher Systemlast, ohne die Erfolgswahrscheinlichkeit zu erhöhen.

Die Abgrenzung zwischen beiden Fehlerarten ist insbesondere für die automatisierte Fehlerbehandlung relevant. Während transiente Fehler häufig eine erneute Ausführung rechtfertigen, sollten permanente Fehler möglichst früh erkannt und gesondert behandelt werden.

Eine besondere Rolle spielen zudem Timeout-Fehler. In verteilten Systemen werden Timeouts verwendet, um potenzielle Ausfälle zu erkennen. Dabei kann jedoch nicht eindeutig unterschieden werden, ob tatsächlich ein Fehler vorliegt oder lediglich eine ungewöhnlich langsame Antwort empfangen wurde. Ein Timeout signalisiert lediglich, dass eine erwartete Antwort nicht innerhalb eines definierten Zeitfensters eingetroffen ist und lässt somit keine Aussage über eine Ursache zu (Bass 2022, Kap. 17.2 Abschnitt „Failure in the Cloud“).

Diese Fehlerarten sind für das Background-Job-Processing-Systeme von besonderer Bedeutung, da sie die Grundlage für die im folgenden Abschnitt vorgestellten Retry- und Backoff-Strategien bilden.

1.3.2 Retry-/Backoff-Strategien

(Newman 2026, Kap. 3 Abschnitt „How Many Retries Are Appropriate?“) (Newman 2026, Kap. 3 Abschnitt „Delays Between Retries“)

Wenn der Fehler nur temporär ist, kann es Sinn machen, die fehler verursachende Aktion zu wiederholen. Wichtig dafür ist eine Klassifikation der Fehler. (Newman 2026, Kap. 3) Daneben muss das System unterscheiden können, ob ein Job tatsächlich noch aktiv bearbeitet wird oder ob er lediglich in einem Zwischenzustand verblieben ist, weil der Worker während der Ausführung unvorhergesehen terminiert ist. Ein weiterer Grundpfeiler des Background Job-Processing ist daher die systematische Fehlerbehandlung und Wiederaufnahme (Kleppmann 2017, Kap. 11 Abschnitt „Acknowledgments and redelivery“). Da Teilausfälle dazu führen können, dass unklar ist, ob ein Verarbeitungsschritt

bereits erfolgreich abgeschlossen wurde, wird mit Wiederholungsmechanismen gearbeitet. Hierbei ist eine exakte Fehlerklassifikation relevant, um transiente Fehler (z. B. kurzzeitige Nichterreichbarkeit eines externen Dienstes) über zeitlich gestaffelte Retry-Strategien mit Backoff zu überbrücken und eine Systemüberlastung zu vermeiden, während permanente Fehler direkt zu einem kontrollierten Abbruch führen (Newman 2026, Kap. 3).

(Microsoft Corporation (Hrsg.) 2026a, Abschnitt „Reliability consideration“—>letzter Punkt) Retries sollten ausschließlich für Fehler durchgeführt werden, deren Ursache außerhalb der eigentlichen Geschäftslogik liegt und deren Fortbestand nicht dauerhaft erwartet wird. Hierzu zählen insbesondere Kommunikations-, Verfügbarkeits- und Nebenläufigkeitsprobleme. Fehler, die auf fehlerhaften Eingabedaten, Verletzungen fachlicher Regeln oder Programmierfehlern beruhen, sollten dagegen unmittelbar als endgültig fehlgeschlagen behandelt werden.

circuite breaker pattern

1.3.3 Idempotenz

Wichtig für exactly-once Semantik (Newman 2026, Kap. 8, Abschnitt „Types Of Delivery Guarantees“) (Kleppmann 2017, Kap.11 -> Atomic commit revisited & Idempotence) (Richardson 2018, Kap.3.3.6) (Richardson 2018, Kap.6.1.6) (Newman 2026, Kap. 3 Abschnitt „Idempotency“) (Hohpe 2005, Kap. 10 Abschnitt „Idempotent Receiver“) Message Broker, um doppelte Nachrichten zu Eliminieren (Richardson 2018, Kap.3.3.6) Einen Message Consumer Idempotent machen (Richardson 2018, Kap.6.1.6) Die Wiederaufnahme darf jedoch nicht zu inkonsistenten Ergebnissen führen, wenn eine Verarbeitung bereits teilweise erfolgt ist und somit Duplikate oder Wiederholungen auftreten (Newman 2026, Kap. 3 Abschnitt „Idempotency“). Statusänderungen und fachliche Datenänderungen müssen folglich über klar definierte Transaktionsgrenzen abgesichert werden, um eine konsistente Weiterbearbeitung nach einem Crash zu gewährleisten (Kleppmann 2017, Kap. 7).

1.3.4 Recovery

Dein „remote datastore“ ist praktisch deine PostgreSQL-Jobtabelle (persistenter Job-Store). Das entspricht dem Ansatz „State remote halten“, was Recovery vereinfacht, aber Laufzeit/Contention beeinflusst. Dein Worker hat vermutlich lokalen, transienten Zustand während der Verarbeitung (in-memory). Der muss nach Crash nicht „rekonstruiert“ werden, sondern du brauchst Mechanismen wie Lease/Timeout + Retry, um Jobs wieder aufzunehmen. „Replay state from input“ entspricht bei dir am ehesten: Job erneut ausführen (at-least-once) – und genau deshalb brauchst du Idempotenz, weil „replay“ bei Side-Effects sonst doppelte Ergebnisse erzeugt. (Kleppmann 2017, Kap.11 Rebuilding a state failure) (Bass 2022, Kap. 4 Abschnitt „Recover from Faults“)

1.4 Zustandsmodellierung mittels State Machines

Eine architektonische Option besteht darin, den aktuellen Zustand in einem externen Datenspeicher (*Remote Datastore*) zu verwalten. (Newman 2026, Kap. 7 Abschnitt „Stateful vs Stateless Processing“)

1.5 Stand der Technik

Für die praktische Umsetzung existieren unterschiedliche Frameworks, die einzelne Aspekte von Background Job-Processing adressieren. Quartz Scheduler bietet trigger- bzw. zeitgesteuertes Scheduling und unterstützt Persistenz sowie Clustering zur koordinierten Ausführung.[Terracotta Inc. 2026] Spring Batch stellt eine Infrastruktur für Batch-Verarbeitung bereit, insbesondere mit Job-/Step-Modell und Persistenz der Ausführungshistorie.[Broadcom Inc. 2026] JobRunr bietet datenbankgestütztes Background Job-Processing in Java mit integriertem Retry-Handling.[Rosoco BV 2026] Temporal stellt eine Workflow-Engine für langlebige, robuste Ausführung bereit, erfordert jedoch einen eigenständigen Serverbetrieb und damit zusätzliche Betriebskomplexität.[Temporal Technologies Inc. 2026] Diese Lösungen verdeutlichen, dass „Background Job-Processing“ in der Praxis nicht allein durch eine Bibliothek gelöst wird, sondern durch Architekturentscheidungen zur Persistenz, zur Nebenläufigkeitskontrolle sowie zur Fehlerbehandlung und Wiederaufnahme.[Kleppmann 2017] Die konkrete Wahl und Kombination dieser Mechanismen hängt von den gewünschten Zusicherungen und den betrachteten Fehlerszenarien ab.[Kleppmann 2017]

hier noch Message Queues ansprechen und warum diese die behandelte Problematik nicht vollständig lösen.

Eventuate Framework (Chris Richardson) als industriellen Referenzpunkt integrieren.

- Konzeptionelle Parallele: Nutzt das Transactional Outbox Pattern zur atomaren Kopplung von Fachdaten und Nachricht/Job in einer gemeinsamen ACID-Transaktion (PostgreSQL).
- Technischer Unterschied: Eventuate nutzt primär Change Data Capture (CDC / Log Tailer), um Events an einen externen Broker (z. B. Kafka) zu pushen.
- Abgrenzung meiner Arbeit: Ich evaluiere das schlankere, rein datenbankgestützte Pull-Modell via SQL-Polling und ‘SKIP LOCKED’, um die Betriebskomplexität eines separaten Brokers einzusparen.

Literaturverzeichnis

- Bass, Len (2022). *Software architecture in practice*. Unter Mitarb. von Paul Clements und Rick Kazman. Fourth edition. SEI series in software engineering. Boston: Addison-Wesley.
- Hohpe, Gregor (2005). *Enterprise integration patterns designing, building, and deploying messaging solutions*. 5. print. Boston [u.a.: Addison-Wesley.
- Kleppmann, Martin (2017). *Designing data-intensive applications: the big ideas behind reliable, scalable, and maintainable systems*. First edition. Beijing, [China: O'Reilly.
- Kudraß, Thomas (5. März 2015). *Taschenbuch Datenbanken*. Hrsg. von Kudraß Thomas. Carl Hanser Verlag GmbH & Co. KG. ISBN: 978-3-446-43508-7. DOI: 10.3139/9783446440265.fm. URL: <https://www.hanser-elibrary.com/doi/10.3139/9783446440265.fm> (besucht am 21.06.2026).
- Microsoft Corporation (Hrsg.) (31. März 2026a). *Best Practices for Background Jobs - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/best-practices/background-jobs> (besucht am 17.06.2026).
- (12. Juni 2026b). *Queue-Based Load Leveling Pattern - Azure Architecture Center*. URL: <https://learn.microsoft.com/en-us/azure/architecture/patterns/queue-based-load-leveling> (besucht am 17.06.2026).
- Newman, Sam (Sep. 2026). *Building Resilient Distributed Systems*. O'Reilly. URL: <https://learning.oreilly.com/library/view/building-resilient-distributed/9781098163532/> (besucht am 24.05.2026).
- Richardson, Chris (2018). *Microservices patterns*. Software development. Shelter Island: Manning.
- Saake, Gunter, Andreas Heuer und Kai-Uwe Sattler (Juni 2018). *Datenbanken – Konzepte und Sprachen*. mitp Verlags GmbH & Co KG. ISBN: 978-3-95845-778-2. URL: <https://learning.oreilly.com/library/view/datenbanken-konzepte/9783958457782/> (besucht am 19.06.2026).
- The PostgreSQL Global Development Group (Hrsg.) (14. Mai 2026). *PostgreSQL 18.4 Documentation*. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/18/index.html> (besucht am 21.06.2026).